

Santo Pegasus Soluciones: Guía Oficial de Ingeniería Front-end

Versión del Documento: 1.0.0

Última Actualización: Octubre de 2025

Departamento: Ingeniería de Software / Chapter de Front-end

Tabla de Contenidos

1. Introducción y Filosofía del Front-end en Santo Pegasus
2. Principios Fundamentales de Ingeniería
3. Stack Tecnológico Estándar
4. Arquitectura de Componentes
5. Gestión del Estado
6. Consumo de APIs
7. Estándares de Código
8. Estilo y Design System
9. Gestión de Formularios
10. Estrategia y Cobertura de Pruebas
11. Proceso de Code Review
12. Control de Versiones
13. Performance y Web Vitals
14. Seguridad en el Front-end
15. CI/CD Front-end

1. Introducción y Filosofía del Front-end en Santo Pegasus

1.1 Presentación Institucional

Bienvenido a la Guía Oficial de Ingeniería Front-end de Santo Pegasus Soluciones. Este documento constituye la fuente de la verdad para todos los desarrolladores que integran el Chapter de Front-end, y su consulta es de carácter obligatorio para cualquier proyecto nuevo o refactorización significativa de interfaces de usuario.

Esta guía fue concebida para existir en perfecta coherencia con la Guía Oficial de Ingeniería Back-end (versión 2.4.0, Octubre de 2025), con la cual comparte filosofía, valores de ingeniería y estándares de proceso. El lector que ya domine el documento de back-end encontrará en estas páginas un lenguaje familiar, aunque aplicado a las particularidades y desafíos propios de la capa de presentación.

Santo Pegasus Soluciones construye productos digitales de alta complejidad, cuyas interfaces de usuario son el principal punto de contacto entre la tecnología que desarrollamos y las personas que la utilizan. La excelencia en el front-end no es un ornamento: es un requisito de negocio.

1.2 Filosofía del Front-end

En Santo Pegasus, entendemos el front-end como una disciplina de ingeniería de software completa, sujeta a los mismos criterios de rigor técnico, legibilidad, seguridad y sostenibilidad que cualquier otra capa del sistema. Rechazamos la concepción que trata el código de interfaz como un conjunto de scripts auxiliares; en cambio, lo tratamos como un sistema distribuido de componentes que debe ser diseñado, probado, versionado y mantenido con la misma disciplina que cualquier microservicio de nuestro ecosistema back-end.

Los tres pilares filosóficos que guían todas nuestras decisiones de front-end son:

- **Experiencia del Usuario como Métrica Técnica:** La performance percibida por el usuario final —medida a través de Web Vitals— es un criterio de aceptación técnica, no una preocupación exclusiva del área de producto.
- **Componentes como Contratos:** Cada componente expone una interfaz pública (sus props) que debe ser tan cuidadosamente diseñada como los endpoints de una API REST. Un componente bien diseñado es predecible, reutilizable y testeable de forma aislada.
- **Seguridad en la Capa de Presentación:** El front-end no es un entorno confiable. Todo dato que proviene del servidor debe ser tratado con desconfianza; todo dato introducido por el usuario debe ser validado y saneado. La seguridad es una responsabilidad compartida entre todas las capas del sistema.

1.3 Alcance y Aplicabilidad

Las directrices de este documento son de aplicación obligatoria para:

- Todos los proyectos nuevos iniciados a partir de Octubre de 2025.
- Cualquier refactorización que afecte más del 30% de la base de código de un proyecto existente.
- Todo desarrollador del Chapter de Front-end, independientemente de su nivel de seniority.

Su aplicación es fuertemente recomendada para proyectos legacy en mantenimiento activo, donde la adopción gradual es preferible a la no adopción.

1.4 Actualización y Revisión

Este documento es un artefacto vivo. Cualquier miembro del Chapter de Front-end puede proponer modificaciones a través de un Pull Request dirigido al repositorio `eng-guidelines`, siguiendo el proceso de Code Review estándar descrito en la Sección 11. Las actualizaciones deben ser aprobadas por al menos dos miembros Senior del Chapter y por el Tech Lead responsable.

2. Principios Fundamentales de Ingeniería

Todo desarrollador del Chapter de Front-end debe tener los siguientes principios profundamente internalizados. Su aplicación no es opcional: es la base sobre la cual se construye código sostenible, escalable y colaborativo.

2.1 SOLID Aplicado al Front-end

Los principios SOLID, aunque originados en el contexto de la orientación a objetos, poseen una traducción directa y poderosa en el desarrollo de interfaces modernas basadas en componentes.

S — Single Responsibility Principle (SRP)

Un componente debe tener una única razón para cambiar. En la práctica, esto significa que un componente no debe simultáneamente obtener datos de la API, transformarlos, manejar el estado de carga y renderizar la interfaz. Cada una de estas responsabilidades debe estar delegada a la capa correspondiente: los hooks de datos, los transformadores de estado y los componentes de presentación.

Incorrecto:

```
``tsx
// ProductCard hace demasiado
const ProductCard = ({ productId }: { productId: string }) => {
  const [product, setProduct] = useState(null);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    fetch(`/api/products/${productId}`)
      .then(res => res.json())
      .then(data => {
        setProduct(data);
        setLoading(false);
      });
  }, [productId]);
```

```

    if (loading) return <Spinner />;
    return <div>{product?.name}</div>;
  };
  ...

```

Correcto:

```

````tsx
// Responsabilidades separadas
// hook: useProduct.ts
export const useProduct = (productId: string) =>
 useQuery({ queryKey: ['product', productId], queryFn: () => fetchProduct(productId) });
// componente de presentación: ProductCard.tsx
export const ProductCard = ({ product }: { product: Product }) => (
 <div>{product.name}</div>
);
// contenedor: ProductCardContainer.tsx
export const ProductCardContainer = ({ productId }: { productId: string }) => {
 const { data, isLoading } = useProduct(productId);
 if (isLoading) return <Spinner />;
 return <ProductCard product={data!} />;
};
...

```

### #### O — Open/Closed Principle (OCP)

**Los componentes deben estar abiertos para extensión y cerrados para modificación. En React, esto se implementa a través de composición y del patrón de props como punto de extensión, evitando modificar el cuerpo interno de un componente para agregar nuevos comportamientos.**

### #### L — Liskov Substitution Principle (LSP)

*Un componente especializado debe poder sustituir a su componente base sin alterar el comportamiento esperado por el sistema. En términos prácticos, si un `PrimaryButton` extiende a `Button`, debe respetar y soportar todas las props del componente padre.*

### #### I — Interface Segregation Principle (ISP)

*Las interfaces de props de los componentes deben ser específicas y cohesivas. Un componente no debe estar obligado a recibir props que no utiliza. Prefiera la composición de interfaces pequeñas sobre la creación de un único objeto de props monolítico.*

### #### D — Dependency Inversion Principle (DIP)

Los componentes de alto nivel no deben depender de implementaciones concretas. En React, esto se expresa a través de hooks de abstracción (por ejemplo, `useUserRepository`) que pueden ser fácilmente sustituidos por mocks en los tests, sin que el componente conozca la fuente real de los datos.

## 2.2 DRY — Don't Repeat Yourself

La duplicación de código en el front-end es uno de los principales generadores de deuda técnica. La creación de un nuevo componente, hook o función utilitaria debe siempre ir precedida de una búsqueda en la base de código existente. Cuando se identifique lógica duplicada en dos o más lugares, la refactorización hacia una abstracción compartida es obligatoria.

El directorio `src/components/ui/` (componentes del Design System) y `src/hooks/` son los repositorios canónicos de código reutilizable del proyecto.

## 2.3 KISS — Keep It Simple, Stupid

La complejidad innecesaria es el mayor enemigo de la mantenibilidad. Ante la elección entre una solución simple y una solución "elegante" pero compleja, Santo Pegasus elige siempre la simplicidad. El código que cualquier desarrollador de nivel Pleno puede leer y entender en menos de cinco minutos es preferible al código que requiere conocimiento especializado de patrones avanzados.

## 2.4 YAGNI — You Aren't Gonna Need It

No construya abstracciones, componentes genéricos o sistemas de configuración para casos de uso hipotéticos que aún no existen. La arquitectura debe responder a necesidades reales y presentes. La sobre-ingeniería prematura es tan perjudicial como la ausencia de arquitectura. Los componentes deben ser generalizados únicamente cuando la necesidad de reutilización se haya demostrado en la práctica.

---

## 3. Stack Tecnológico Estándar

La coherencia del stack tecnológico es fundamental para reducir la fricción entre equipos, facilitar la rotación de desarrolladores entre proyectos y garantizar que las soluciones de CI/CD sean aplicables transversalmente.

### 3.1 Tabla del Stack Principal

Categoría	Tecnología	Versión Mínima	Justificativa
Lenguaje	TypeScript	5.0+	Tipado estático, integración con APIs back-end

Framework UI	React	18+	Concurrent features, ecosistema maduro
Meta-framework	Next.js	14+	SSR/SSG, App Router, RSC
Bundler	Vite	5+	Build rápido para SPA, HMR eficiente
Estado Global	Zustand	4+	Ligereza, sin boilerplate
Estado del Servidor	TanStack Query	5+	Cache, revalidación, sincronización
Formularios	React Hook Form	7+	Performance, integración con Zod
Validación	Zod	3+	Schemas tipados, integración con RHF
HTTP Client	Axios	1.6+	Interceptores, manejo centralizado
Estilos	Tailwind CSS	3+	Utility-first, consistencia de tokens
Componentes Base	shadcn/ui	Latest	Accesible, headless, personalizable
Testing Unitario	Vitest	1+	Nativo Vite, API compatible con Jest
Testing Componentes	Testing Library	14+	Enfoque en comportamiento de usuario
Testing E2E	Playwright	1.40+	Multi-browser, CI-ready
Linting	ESLint	8+	Calidad y consistencia de código
Formateo	Prettier	3+	Formato uniforme, sin debates

### 3.2 React 18+ y Concurrent Features

El uso de React 18+ es obligatorio en todos los proyectos nuevos. Las siguientes características deben ser adoptadas de forma apropiada:

- **`Suspense`**: Utilizado para manejar estados de carga de componentes asíncronos, en combinación con TanStack Query.
- **`useTransition` y `startTransition`**: Adoptados para marcar actualizaciones de estado no urgentes, manteniendo la interfaz responsiva durante renderizados costosos.

- **Server Components (Next.js):** En proyectos Next.js, se debe privilegiar el uso de React Server Components para contenido estático o fetching inicial de datos, reduciendo el JavaScript enviado al cliente.

### 3.3 Next.js para Proyectos con SSR/SSG

Next.js es el meta-framework obligatorio cuando el proyecto requiera:

- **SEO: Páginas de marketing, catálogos de productos, blogs institucionales.**
- **Performance de carga inicial: Aplicaciones con usuarios en redes lentas o dispositivos de bajo rendimiento.**
- **Server-Side Rendering: Datos que deben ser pre-renderizados en el servidor por razones de seguridad o consistencia.**

Para aplicaciones puramente internas (dashboards administrativos, herramientas de gestión sin requisitos de SEO), el uso de Vite + React como SPA es la opción estándar y preferida por su simplicidad operativa.

### 3.4 Vite como Bundler para SPAs

La configuración de Vite debe mantenerse en el archivo `vite.config.ts` en la raíz del proyecto. Las personalizaciones deben ser documentadas con comentarios que expliquen la justificativa técnica de cada opción no estándar.

```
``typescript
// vite.config.ts
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import path from 'path';

export default defineConfig({
 plugins: [react()],
 resolve: {
 alias: {
 '@': path.resolve(__dirname, './src'),
 },
 },
 build: {
 // Code splitting automático por ruta
 rollupOptions: {
 output: {
 manualChunks: {
 vendor: ['react', 'react-dom'],
 query: ['@tanstack/react-query'],
 },
 },
 },
 },
});
```

```

 },
 },
 },
 },
 });
 ...

```

## 4. Arquitectura de Componentes

### 4.1 Atomic Design

**Santo Pegasus adopta la metodología Atomic Design como sistema de clasificación y organización de componentes.** Esta metodología, propuesta por Brad Frost, establece una jerarquía clara que facilita la reutilización, la consistencia visual y la comunicación entre desarrolladores y diseñadores.

#### #### 4.1.1 Atoms (Átomos)

Son los bloques de construcción más elementales de la interfaz. No pueden descomponerse en componentes más pequeños sin perder su significado funcional. Los átomos no contienen lógica de negocio ni estado derivado de APIs.

**Ejemplos: `Button`, `Input`, `Label`, `Icon`, `Badge`, `Avatar`, `Spinner`.**

```

``tsx
// src/components/atoms/Button/Button.tsx
interface ButtonProps extends React.ButtonHTMLAttributes<HTMLButtonElement> {
 variant?: 'primary' | 'secondary' | 'destructive' | 'ghost';
 size?: 'sm' | 'md' | 'lg';
 isLoading?: boolean;
}
export const Button = ({
 variant = 'primary',
 size = 'md',
 isLoading = false,
 children,
 disabled,
 ...props
}: ButtonProps) => (
 <button
 className={cn(buttonVariants({ variant, size }))}
 disabled={disabled || isLoading}

```



```

 aria-busy={isLoading}
 {...props}
 >
 {isLoading ? <Spinner size="sm" /> : children}
 </button>
);
...

```

#### #### 4.1.2 Molecules (Moléculas)

Son combinaciones de dos o más átomos que forman una unidad funcional con un propósito único y bien definido. Las moléculas pueden contener estado local simple (por ejemplo, el estado de foco de un campo de búsqueda).

**Ejemplos:** ``SearchField`` (Input + Icon + Button), ``FormField`` (Label + Input + ErrorMessage), ``UserAvatarWithName`` (Avatar + Text).

#### #### 4.1.3 Organisms (Organismos)

**Son secciones complejas y autónomas de la interfaz, formadas por la composición de moléculas y átomos. Los organismos pueden contener lógica de presentación más compleja y representan regiones reconocibles de la UI. Sin embargo, no deben realizar llamadas directas a APIs; esa responsabilidad pertenece a sus contenedores.**

**Ejemplos:** ``ProductTable``, ``NavigationHeader``, ``UserProfileCard``, ``CheckoutSummary``.

#### #### 4.1.4 Templates (Plantillas)

*Definen la estructura y el layout de una página, estableciendo dónde se posicionarán los organismos, moléculas y átomos. Los templates trabajan con datos de marcador de posición (placeholder), son agnósticos del contenido real y representan el esqueleto visual de una ruta.*

**Ejemplos:** ``DashboardTemplate``, ``AuthTemplate``, ``TwoColumnLayout``.

#### #### 4.1.5 Pages (Páginas)

Son la instanciación concreta de un Template con datos reales. Las Pages son la capa de composición final, donde se conectan los contenedores de datos con los componentes de presentación. En proyectos Next.js, corresponden a los archivos dentro del directorio ``app/`` o ``pages/``.

## 4.2 Componentes de Presentación vs. Contenedores

**La separación entre componentes de presentación (dumb components) y contenedores (smart components) es una directriz de arquitectura de aplicación obligatoria en Santo Pegasus.**

Característica	Componente de Presentación	Contenedor
Responsabilidad	Renderizar UI a partir de props	Obtener datos, manejar estado
Llamadas a API	Nunca	Vía hooks de datos

Estado global	No accede	Accede vía Zustand/Query
Testabilidad	Alta (solo recibe props)	Media (requiere mocks)
Reutilización	Alta	Baja (acoplado al dominio)
Sufijo de archivo	`ProductCard.tsx`	`ProductCardContainer.tsx`

## 5. Gestión del Estado

### 5.1 Taxonomía del Estado

Antes de elegir una solución de estado, es obligatorio clasificar la naturaleza del estado a gestionar:

Tipo de Estado	Descripción	Solución Recomendada
Estado Local	UI efímera (modal abierto, tab activo)	`useState`, `useReducer`
Estado del Servidor	Datos provenientes de APIs	TanStack Query
Estado Global del Cliente	Auth, preferencias, tema, carrito	Zustand
Estado de URL	Filtros, paginación, búsqueda	`useSearchParams`
Estado de Formulario	Inputs, validaciones, envío	React Hook Form

### 5.2 Zustand para Estado Global del Cliente

*Zustand es la solución estándar para estado global que no proviene del servidor. Su adopción está justificada por su mínimo boilerplate, ausencia de providers anidados y excelente integración con TypeScript.*

#### #### 5.2.1 Reglas de Uso de Zustand

- 1. Una store por dominio de negocio:** No se debe crear una única store global monolítica. Cada dominio (autenticación, UI global, carrito, notificaciones) debe tener su propia store.
- 2. Interfaz tipada obligatoria:** Toda store debe definir explícitamente su interfaz TypeScript.
- 3. Acciones junto al estado:** Las funciones que modifican el estado deben definirse dentro de la propia store.
- 4. Selectores para prevenir re-renders:** Siempre utilice selectores específicos al consumir la store.

```
``typescript
// src/store/auth.store.ts
interface AuthState {
```

```

user: User | null;
accessToken: string | null;
isAuthenticated: boolean;
setUser: (user: User, token: string) => void;
clearAuth: () => void;
}

export const useAuthStore = create<AuthState>((set) => ({
 user: null,
 accessToken: null,
 isAuthenticated: false,
 setUser: (user, accessToken) =>
 set({ user, accessToken, isAuthenticated: true }),
 clearAuth: () =>
 set({ user: null, accessToken: null, isAuthenticated: false }),
}));

// Uso con selector (previene re-renders innecesarios)
const user = useAuthStore((state) => state.user);
...

```

## 5.3 TanStack Query para Estado del Servidor

TanStack Query (anteriormente React Query) es la solución obligatoria para todo estado que proviene de APIs externas. Su adopción elimina la necesidad de `useEffect` para fetch de datos y proporciona cache, revalidación, estados de carga y error de forma automática.

### #### 5.3.1 Reglas de Uso de TanStack Query

**1. Nunca usar `useEffect` para fetch de datos: Todo consumo de API debe realizarse a través de `useQuery` o `useMutation`.**

**2. Query Keys estructuradas: Las query keys deben seguir una estructura jerárquica consistente que refleje el dominio y los parámetros.**

**3. Hooks de dominio: Los calls a `useQuery` / `useMutation` deben encapsularse en hooks de dominio, no usarse directamente en los componentes.**

```

``typescript
// src/hooks/queries/useProducts.ts
// Convención de query keys: [entidad, operación, ...parámetros]
export const productKeys = {
 all: ['products'] as const,
 lists: () => [...productKeys.all, 'list'] as const,
 list: (filters: ProductFilters) => [...productKeys.lists(), filters] as const,

```

```

 detail: (id: string) => [...productKeys.all, 'detail', id] as const,
 };
export const useProducts = (filters: ProductFilters) =>
 useQuery({
 queryKey: productKeys.list(filters),
 queryFn: () => productService.getAll(filters),
 staleTime: 5 * 60 * 1000, // 5 minutos
 });
export const useProduct = (id: string) =>
 useQuery({
 queryKey: productKeys.detail(id),
 queryFn: () => productService.getById(id),
 enabled: !!id,
 });
export const useCreateProduct = () => {
 const queryClient = useQueryClient();
 return useMutation({
 mutationFn: productService.create,
 onSuccess: () => {
 // Invalida el cache de la lista tras crear
 queryClient.invalidateQueries({ queryKey: productKeys.lists() });
 },
 });
};
...

```

## 6. Consumo de APIs

### 6.1 Cliente HTTP Centralizado

Todo consumo de las APIs de Santo Pegasus debe realizarse a través de una instancia centralizada de Axios, configurada con interceptores para el manejo automático de autenticación, refresh de tokens y errores globales. Queda terminantemente prohibido instanciar Axios directamente en componentes o hooks de dominio.

### 6.2 Configuración del Cliente Axios

```

``typescript
// src/lib/api/client.ts
```

```

import axios, { AxiosInstance, InternalAxiosRequestConfig } from 'axios';
import { useAuthStore } from '@store/auth.store';
const apiClient: AxiosInstance = axios.create({
 baseURL: import.meta.env.VITE_API_BASE_URL,
 timeout: 15000,
 headers: {
 'Content-Type': 'application/json',
 },
 // Las cookies httpOnly son enviadas automáticamente
 withCredentials: true,
});
// Interceptor de Request: inyecta el token JWT
apiClient.interceptors.request.use(
 (config: InternalAxiosRequestConfig) => {
 const token = useAuthStore.getState().accessToken;
 if (token) {
 config.headers.Authorization = `Bearer ${token}`;
 }
 return config;
 },
 (error) => Promise.reject(error)
);
// Interceptor de Response: manejo de errores y refresh token
apiClient.interceptors.response.use(
 (response) => response,
 async (error) => {
 const originalRequest = error.config;
 if (error.response?.status === 401 && !originalRequest._retry) {
 originalRequest._retry = true;
 try {
 // Refresh token via cookie httpOnly — no expone el token en JS
 const { data } = await axios.post('/auth/refresh', {}, { withCredentials: true });
 useAuthStore.getState().setUser(data.user, data.accessToken);
 originalRequest.headers.Authorization = `Bearer ${data.accessToken}`;
 return apiClient(originalRequest);
 } catch {
 useAuthStore.getState().clearAuth();
 }
 }
 }
);

```

```

 window.location.href = '/login';
 }
}
// Manejo global de errores por código HTTP
handleHttpError(error.response?.status);
return Promise.reject(error);
}
);
...

```

## 6.3 Tipado Fuerte de Requests y Responses

Todo servicio de dominio debe tipar explícitamente sus parámetros de entrada y sus respuestas. Nunca se debe utilizar el tipo `any` en operaciones de red.

```

``typescript
// src/lib/api/services/product.service.ts
import { apiClient } from '@lib/api/client';
import type { Product, CreateProductDto, ProductFilters, PaginatedResponse } from
'@/types/product.types';

export const productService = {
 getAll: async (filters: ProductFilters): Promise<PaginatedResponse<Product>> => {
 const { data } = await apiClient.get<PaginatedResponse<Product>>('/products', {
 params: filters,
 });
 return data;
 },
 getById: async (id: string): Promise<Product> => {
 const { data } = await apiClient.get<Product>(`/products/${id}`);
 return data;
 },
 create: async (payload: CreateProductDto): Promise<Product> => {
 const { data } = await apiClient.post<Product>('/products', payload);
 return data;
 },
};
...

```

## 6.4 Manejo Global de Errores HTTP

La función `handleHttpError` debe cubrir los códigos de respuesta estándar y disparar notificaciones globales usando el sistema de toast del Design System.

```
``typescript
// src/lib/api/error-handler.ts
import { toast } from '@components/ui/toast';
const HTTP_ERROR_MESSAGES: Record<number, string> = {
 400: 'Solicitud inválida. Verifique los datos enviados.',
 403: 'Usted no tiene permisos para realizar esta acción.',
 404: 'El recurso solicitado no fue encontrado.',
 409: 'Conflicto: el recurso ya existe o está en uso.',
 422: 'Los datos enviados no pudieron ser procesados.',
 500: 'Error interno del servidor. Contacte al soporte técnico.',
 503: 'Servicio temporalmente no disponible. Intente nuevamente.',
};
export const handleHttpError = (status?: number): void => {
 if (!status) return;
 const message = HTTP_ERROR_MESSAGES[status] ?? 'Ocurrió un error inesperado.';
 toast.error(message);
};
...

```

## 7. Estándares de Código

### 7.1 Nomenclatura de Archivos y Directorios

La consistencia en la nomenclatura es obligatoria y será verificada en el proceso de Code Review.

Tipo de Artefacto	Convención	Ejemplo
Componentes React	PascalCase	<code>ProductCard.tsx</code>
Custom Hooks	camelCase con prefijo <code>use</code>	<code>useProductList.ts</code>
Funciones utilitarias	camelCase	<code>formatCurrency.ts</code>
Servicios de API	camelCase con sufijo <code>.service</code>	<code>product.service.ts</code>
Stores Zustand	camelCase con sufijo <code>.store</code>	<code>auth.store.ts</code>
Tipos e Interfaces	PascalCase con sufijo <code>.types</code>	<code>product.types.ts</code>
Constantes	SCREAMING_SNAKE_CASE	<code>API_ENDPOINTS.ts</code>

Archivos de Test	Mismo nombre + `.test` o `.spec`	`ProductCard.test.tsx`
Directorios	kebab-case	`product-card/`

## 7.2 Estructura de Carpetas del Proyecto

...

src/

app/ # Rutas (Next.js App Router) o pages (Vite)

components/

atoms/ # Componentes elementales del Design System

molecules/ # Composiciones de átomos

organisms/ # Secciones complejas de UI

templates/ # Layouts de página

ui/ # Re-exportaciones de shadcn/ui

hooks/

queries/ # Hooks de TanStack Query por dominio

mutations/ # Hooks de useMutation por dominio

lib/

api/ # Cliente Axios, servicios, error-handler

utils/ # Funciones utilitarias puras

store/ # Stores de Zustand

types/ # Interfaces y tipos globales de TypeScript

styles/ # Configuraciones globales de Tailwind y CSS

tests/

unit/ # Tests unitarios de utils y hooks

components/ # Tests de componentes con Testing Library

e2e/ # Tests Playwright

constants/ # Constantes globales de la aplicación

...

## 7.3 ESLint — Configuración Obligatoria

El proyecto debe incluir una configuración ESLint que extienda las siguientes reglas base:

```
``json
```

```
{
```

```
"extends": [
```

```
"eslint:recommended",
```

```
"plugin:@typescript-eslint/recommended",
```

```
"plugin:react-hooks/recommended",
```



```

 "plugin:jsx-a11y/recommended",
 "plugin:import/typescript",
 "prettier"
],
 "rules": {
 "@typescript-eslint/no-explicit-any": "error",
 "@typescript-eslint/no-unused-vars": "error",
 "react-hooks/exhaustive-deps": "error",
 "import/order": ["error", { "newlines-between": "always" }],
 "no-console": ["warn", { "allow": ["error"] }]
 }
}
...

```

## 7.4 Prettier — Configuración Estándar

```

```.json
{
  "semi": true,
  "trailingComma": "all",
  "singleQuote": true,
  "printWidth": 100,
  "tabWidth": 2,
  "arrowParens": "always"
}
...

```

El formateo automático con Prettier debe ejecutarse como pre-commit hook mediante Husky + lint-staged, garantizando que ningún código mal formateado llegue al repositorio.

8. Estilo y Design System

8.1 Tailwind CSS como Sistema de Tokens

Tailwind CSS es el sistema de estilos estándar de Santo Pegasus. Su filosofía utility-first elimina los problemas de especificidad de CSS, facilita la revisión de estilos en Code Review y garantiza la consistencia visual a través de tokens predefinidos.

La personalización del tema debe realizarse exclusivamente en `tailwind.config.ts`, donde se definen los tokens de diseño del proyecto.

```

```.typescript

```

```

// tailwind.config.ts
import type { Config } from 'tailwindcss';

const config: Config = {
 content: ['./src/**/*.ts', './src/**/*.tsx'],
 theme: {
 extend: {
 colors: {
 brand: {
 50: '#f0f4ff',
 500: '#3b5bdb',
 900: '#1a237e',
 },
 neutral: {
 50: '#f8f9fa',
 500: '#868e96',
 900: '#212529',
 },
 success: '#2f9e44',
 warning: '#f59f00',
 error: '#e03131',
 },
 fontFamily: {
 sans: ['Inter', 'system-ui', 'sans-serif'],
 mono: ['JetBrains Mono', 'monospace'],
 },
 spacing: {
 '4.5': '1.125rem',
 '18': '4.5rem',
 },
 },
 },
 plugins: [require('tailwindcss-animate')],
};

export default config;

```

## 8.2 shadcn/ui como Biblioteca de Componentes Base

**shadcn/ui** es la biblioteca de componentes base estándar de Santo Pegasus. Sus componentes son instalados directamente en el repositorio del proyecto (no como dependencia de `node_modules`), lo que permite su personalización completa sin conflictos de versioning.

#### 8.2.1 Reglas de Uso de shadcn/ui

- 1. Los componentes de shadcn/ui deben residir en ``src/components/ui/`` y no deben ser modificados directamente. En su lugar, créese un componente envolvente en la capa de átomos que extienda el comportamiento deseado.
- 2. Todo componente de shadcn/ui utilizado debe tener su accesibilidad verificada conforme a la Sección 8.3.

**8.3 Accesibilidad — WCAG 2.1 Nivel AA**

La conformidad con el estándar WCAG 2.1 Nivel AA es un requisito no negociable en Santo Pegasus. Las siguientes directrices son de aplicación obligatoria:

Criterio	Requisito	Herramienta de Verificación
Contraste de Color	Ratio mínimo 4.5:1 (texto normal), 3:1 (texto grande)	Figma Contrast Plugin, axe DevTools
Navegación por Teclado	Todos los elementos interactivos deben ser accesibles vía Tab	Prueba manual + <code>`eslint-plugin-jsx-a11y`</code>
Textos Alternativos	<code>`alt`</code> descriptivo en todas las imágenes significativas	<code>`eslint-plugin-jsx-a11y`</code>
Labels en Formularios	Cada campo debe tener un <code>&lt;label&gt;</code> asociado	<code>`eslint-plugin-jsx-a11y`</code>
Roles ARIA	Uso correcto de <code>`role`</code> , <code>`aria-label`</code> , <code>`aria-describedby`</code>	axe DevTools, revisión en PR
Gestión de Foco	Modales y drawers deben atrapar el foco correctamente	Prueba manual
Skip Links	Páginas con navegación deben incluir "Saltar al contenido"	Revisión en PR

La regla ``plugin:jsx-a11y/recommended`` en ESLint automatiza la detección de la mayoría de estas violaciones en tiempo de desarrollo.

---

**9. Gestión de Formularios**

**9.1 React Hook Form + Zod**

**La gestión de formularios en Santo Pegasus se basa en la combinación de React Hook Form para el control del estado del formulario y Zod para la definición del esquema de validación. Esta combinación garantiza una única fuente de verdad para la lógica de validación, que además genera tipos TypeScript automáticamente.**

## 9.2 Patrón de Implementación

```
``typescript
// src/types/product.types.ts
import { z } from 'zod';
export const createProductSchema = z.object({
 name: z.string()
 .min(3, 'El nombre debe tener al menos 3 caracteres.')
 .max(100, 'El nombre no puede superar 100 caracteres.'),
 price: z.number()
 .positive('El precio debe ser un valor positivo.')
 .multipleOf(0.01, 'El precio debe tener máximo dos decimales.'),
 categoryId: z.string().uuid('Seleccione una categoría válida.'),
 description: z.string().max(500).optional(),
});
// El tipo se infiere automáticamente del schema
export type CreateProductDto = z.infer<typeof createProductSchema>;
...

``tsx
// src/components/organisms/ProductForm/ProductForm.tsx
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { createProductSchema, CreateProductDto } from '@types/product.types';
import { useCreateProduct } from '@hooks/mutations/useCreateProduct';
export const ProductForm = () => {
 const { mutate: createProduct, isPending } = useCreateProduct();
 const {
 register,
 handleSubmit,
 formState: { errors },
 } = useForm<CreateProductDto>({
 resolver: zodResolver(createProductSchema),
 });
```

```

const onSubmit = (data: CreateProductDto) => createProduct(data);
return (
 <form onSubmit={handleSubmit(onSubmit)} noValidate>
 <FormField
 label="Nombre del Producto"
 error={errors.name?.message}
 >
 <Input
 {...register('name')}
 aria-describedby={errors.name ? 'name-error' : undefined}
 />
 </FormField>
 <Button type="submit" isLoading={isPending}>
 Crear Producto
 </Button>
 </form>
);
};
...

```

## 9.3 Reglas de Validación

**1. Validación en el cliente es una mejora de UX, no de seguridad. Los schemas Zod del front-end deben ser equivalentes a las validaciones del back-end (Bean Validation en Java), pero nunca los reemplazan.**

**2. Mensajes de error en español y orientados al usuario, no al desarrollador.**

**3. `noValidate` en todos los formularios: Deshabilitar la validación nativa del navegador para mantener control total sobre el feedback de error.**

---

## 10. Estrategia y Cobertura de Pruebas

La calidad del código de front-end debe ser asegurada por un enfoque multifacético que cubra la lógica de negocio, el comportamiento de los componentes y los flujos críticos del usuario.

### 10.1 Cobertura Mínima Obligatoria

**La cobertura mínima de 80% en pruebas unitarias es obligatoria y verificada automáticamente en el pipeline de CI/CD. El build falla si la cobertura cae por debajo de este umbral.**

### 10.2 Pirámide de Pruebas

#### 10.2.1 Pruebas Unitarias — Vitest

Enfocadas en lógica pura: funciones utilitarias, transformadores de datos, schemas Zod y hooks de negocio aislados de la UI.

```
``typescript
// src/lib/utils/formatCurrency.test.ts
import { describe, it, expect } from 'vitest';
import { formatCurrency } from './formatCurrency';
describe('formatCurrency', () => {
 it('formatea correctamente un valor positivo en BRL', () => {
 expect(formatCurrency(1999.99, 'BRL')).toBe('R$ 1.999,99');
 });
 it('formatea cero como R$ 0,00', () => {
 expect(formatCurrency(0, 'BRL')).toBe('R$ 0,00');
 });
 it('lanza error con valor negativo', () => {
 expect(() => formatCurrency(-1, 'BRL')).toThrow('Valor inválido');
 });
});
``
```

#### #### 10.2.2 Pruebas de Componentes — Testing Library

**Enfocadas en el comportamiento del usuario, no en los detalles de implementación. Las pruebas deben consultar elementos por roles ARIA, labels y texto visible, jamás por selectores CSS o IDs internos.**

```
``tsx
// src/components/organisms/ProductForm/ProductForm.test.tsx
import { render, screen, waitFor } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { ProductForm } from './ProductForm';
describe('ProductForm', () => {
 it('muestra error de validación al enviar nombre vacío', async () => {
 render(<ProductForm />);
 await userEvent.click(screen.getByRole('button', { name: /crear producto/i }));
 expect(await screen.findByText(/al menos 3 caracteres/i)).toBeInTheDocument();
 });
 it('llama a onSubmit con datos válidos', async () => {
 const mockCreate = vi.fn();
 render(<ProductForm onSubmit={mockCreate} />);
 await userEvent.type(screen.getByLabelText(/nombre/i), 'Producto de prueba');
```

```

 await userEvent.click(screen.getByRole('button', { name: /crear producto/i }));
 await waitFor(() => expect(mockCreate).toHaveBeenCalledTimes());
 });
});
...

```

#### #### 10.2.3 Pruebas E2E — Playwright

Obligatorias para los flujos críticos del negocio. Ejecutadas en el pipeline de CI contra un ambiente de Staging.

##### **Flujos que requieren cobertura E2E obligatoria:**

- Autenticación completa (login, refresh, logout)
- Flujo de checkout / acción principal del negocio
- Operaciones CRUD del recurso central del sistema
- Recuperación de contraseña

```

``typescript
// src/tests/e2e/auth.spec.ts
import { test, expect } from '@playwright/test';
test.describe('Autenticación', () => {
 test('usuario puede iniciar sesión con credenciales válidas', async ({ page }) => {
 await page.goto('/login');
 await page.getByLabel('Correo electrónico').fill('usuario@santopegasus.com');
 await page.getByLabel('Contraseña').fill('Senha@Segura123');
 await page.getByRole('button', { name: /iniciar sesión/i }).click();
 await expect(page).toHaveURL('/dashboard');
 await expect(page.getByRole('heading', { name: /bienvenido/i })).toBeVisible();
 });
});
...

```

### **10.3 Configuración de Vitest**

```

``typescript
// vitest.config.ts
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';
export default defineConfig({
 plugins: [react()],
 test: {
 globals: true,

```

```

environment: 'jsdom',
setupFiles: ['./src/tests/setup.ts'],
coverage: {
 provider: 'v8',
 thresholds: {
 lines: 80,
 functions: 80,
 branches: 80,
 statements: 80,
 },
 exclude: ['src/components/ui/', 'src/types/'],
},
},
});
...

```

## 11. Proceso de Code Review

### 11.1 Reglas para el Autor del PR

- 1. Tamaño máximo del PR:** Todo Pull Request debe tener un máximo de 400 líneas de código modificado, excluyendo archivos de configuración, tipos generados automáticamente y assets. PRs que excedan este límite deben ser divididos en unidades lógicas más pequeñas.
- 2. Auto-revisión obligatoria:** El autor debe revisar su propio PR en la interfaz de GitHub/GitLab antes de solicitar revisión, asegurándose de que no haya código de debug, `console.log` o `TODO` no documentados.
- 3. Template del PR:** Todo PR debe completar el template oficial (ver 11.3).
- 4. CI verde:** No se permite solicitar revisión con el pipeline de CI fallando.
- 5. Screenshots o videos:** Toda alteración visual debe incluir capturas de pantalla comparativas (antes/después) o una grabación de la interacción.

### 11.2 Reglas para el Revisor

- 1. Aprobaciones requeridas:** Todo Pull Request exige al menos 2 aprobaciones de miembros Senior o Semi-Senior antes del merge, siguiendo el mismo criterio establecido en la Guía de Ingeniería Back-end.



**2. Tono y Etiqueta:** La comunicación debe estar siempre pautada por la empatía y la educación. Los comentarios deben estar orientados exclusivamente a la mejora del código. Se recomienda el uso de preguntas sugestivas: "¿Consideraste usar ``useMemo`` aquí para evitar el recálculo?" en lugar de imperativos como "Usa ``useMemo``".

**3. Criterios de Revisión Front-end:** El revisor debe validar:

Criterio	Descripción
Correctitud	El componente se comporta como se espera en todos los estados (loading, error, vacío, datos)
Accesibilidad	Uso correcto de roles ARIA, labels, contraste y navegación por teclado
Performance	Ausencia de renders innecesarios, uso correcto de <code>`useMemo`/`useCallback`</code>
Seguridad	Sin datos sensibles en <code>localStorage</code> , sin XSS, sin <code>`dangerouslySetInnerHTML`</code> no sanitizado
Tipado	Ausencia de <code>`any`</code> , tipos explícitos en todas las funciones públicas
Pruebas	Cobertura adecuada de los casos feliz, error y borde
Guía de Ingeniería	Conformidad con las directrices de este documento

**11.3 Template Oficial del PR**

``markdown

**Descripción**

<!-- ¿Qué cambios fueron realizados en este PR? -->

**Motivación**

<!-- ¿Por qué estos cambios son necesarios? Referencia al ticket/issue. -->

Closes: #[número del issue]

**Cómo Probar**

<!-- Instrucciones paso a paso para validar los cambios manualmente. -->

- 1.
- 2.
- 3.

**Screenshots**

<!-- Para cambios visuales, incluir capturas antes/después. -->

Antes	Después
img	img

## Checklist del Autor

- [ ] Código revisado por el propio autor antes de solicitar review
- [ ] Pruebas unitarias/componentes añadidas o actualizadas
- [ ] Cobertura de pruebas  $\geq 80\%$
- [ ] Accesibilidad verificada (teclado + contraste)
- [ ] Sin ``console.log`` ni código de debug
- [ ] Sin ``any`` en TypeScript
- [ ] Pipeline de CI pasando (lint + tests + build)
- [ ] PR tiene menos de 400 líneas modificadas
- [ ] Screenshots incluidos (si aplica)

...

---

## 12. Control de Versiones

### 12.1 GitFlow

Santo Pegasus adopta el modelo GitFlow de forma idéntica a lo establecido en la Guía de Ingeniería Back-end, garantizando la coherencia del proceso de desarrollo entre todas las capas del sistema.

Rama	Propósito	Merge Origin	Merge Target
<code>`main`</code>	Código en producción	<code>`release/`</code> , <code>`hotfix/`</code>	—
<code>`develop`</code>	Integración de features	<code>`feature/`</code> , <code>`fix/`</code>	<code>`release/`</code>
<code>`feature/`</code>	Nuevas funcionalidades	<code>`develop`</code>	<code>`develop`</code>
<code>`fix/`</code>	Correcciones no urgentes	<code>`develop`</code>	<code>`develop`</code>
<code>`release/`</code>	Preparación para deploy	<code>`develop`</code>	<code>`main`</code> + <code>`develop`</code>
<code>`hotfix/`</code>	Correcciones urgentes en producción	<code>`main`</code>	<code>`main`</code> + <code>`develop`</code>

### 12.2 Conventional Commits

El uso de **Conventional Commits** es obligatorio en todos los proyectos del Chapter de Front-end, siguiendo la misma especificación adoptada en el back-end. El formato es validado automáticamente por Commitlint configurado como commit-msg hook en Husky.

```
...
<tipo>(<escopo>): <descripción corta en imperativo>
[cuerpo opcional]
[footer opcional: BREAKING CHANGE, Closes #123]
...
```

**Tipos permitidos:**

Tipo	Uso
`feat`	Nueva funcionalidad visible para el usuario
`fix`	Corrección de bug
`refactor`	Refactorización sin cambio de comportamiento
`style`	Cambios de formato/estilos sin impacto en lógica
`test`	Adición o corrección de pruebas
`chore`	Tareas de mantenimiento (deps, config)
`docs`	Documentación
`perf`	Mejoras de performance
`ci`	Cambios en pipelines de CI/CD

**Ejemplos:**

```
...
feat(product): añade filtro por categoría en listado de productos
fix(auth): corrige loop infinito en refresh token cuando el server retorna 401
perf/dashboard): implementa lazy loading en tabla de transacciones
test(product-form): añade tests para estado de error en formulario de creación
...

```

### 13. Performance y Web Vitals

#### 13.1 Web Vitals como Criterios de Calidad

Las métricas Core Web Vitals de Google son criterios de aceptación técnica en Santo Pegasus. Ninguna funcionalidad puede ser desplegada a producción si degrada significativamente estas métricas por debajo de los umbrales establecidos.

Métrica	Descripción	Umbral "Good"	Umbral Máximo
LCP (Largest Contentful Paint)	Tiempo de carga del elemento más grande visible	$\leq 2.5s$	$\leq 4.0s$
FID (First Input Delay) / INP	Latencia de respuesta a la primera interacción	$\leq 100ms$	$\leq 300ms$
CLS (Cumulative Layout Shift)	Estabilidad visual del layout durante la carga	$\leq 0.1$	$\leq 0.25$
TTFB (Time to First Byte)	Tiempo de respuesta del servidor	$\leq 800ms$	$\leq 1.8s$
FCP (First Contentful Paint)	Primer renderizado de contenido	$\leq 1.8s$	$\leq 3.0s$

**Las métricas deben ser monitoreadas en producción utilizando herramientas como Vercel Analytics, Google Search Console o Datadog RUM.**

## 13.2 Lazy Loading y Code Splitting

**Code splitting es obligatorio para rutas de la aplicación. Ninguna ruta debe ser incluida en el bundle principal (initial chunk).**

```
``tsx
// Correcto: lazy loading por ruta
import { lazy, Suspense } from 'react';
const ProductsPage = lazy(() => import('@app/products/ProductsPage'));
const DashboardPage = lazy(() => import('@app/dashboard/DashboardPage'));
const SettingsPage = lazy(() => import('@app/settings/SettingsPage'));
export const AppRouter = () => (
 <Suspense fallback={<PageLoadingSpinner />}>
 <Routes>
 <Route path="/products" element={<ProductsPage />} />
 <Route path="/dashboard" element={<DashboardPage />} />
 <Route path="/settings" element={<SettingsPage />} />
 </Routes>
 </Suspense>
);
...

```

## 13.3 Optimización de Imágenes

- **Formato:** Usar siempre WebP como formato primario, con fallback para JPEG/PNG.

- **Dimensiones:** Nunca servir imágenes con dimensiones mayores que su tamaño de display.
- `loading="lazy"`: Obligatorio en todas las imágenes que no estén en el viewport inicial.
- `width` y `height`` explícitos: Obligatorio para prevenir CLS durante la carga.
- **Next.js Image:** En proyectos Next.js, el componente `<Image />` es obligatorio. Queda prohibido el uso de la etiqueta `<img>` nativa para imágenes de contenido.

### 13.4 Memoización y Prevención de Re-renders

El uso de `React.memo``, `useMemo`` y `useCallback`` debe ser justificado y medido, no aplicado de forma prematura. La regla es: primero identificar el problema con las DevTools de React Profiler, luego aplicar la memoización.

```

````tsx
// Uso justificado: componente hijo que recibe callback y re-renderiza en cascada
const ProductRow = React.memo(({ product, onSelect }: ProductRowProps) => (
  <tr onClick={() => onSelect(product.id)}>
    <td>{product.name}</td>
  </tr>
));
// En el padre, el callback debe ser estabilizado
const handleSelect = useCallback((id: string) => {
  setSelectedProductId(id);
}, []); // sin dependencias, siempre la misma referencia
````

```

## 14. Seguridad en el Front-end

### 14.1 Filosofía de Seguridad

El front-end opera en un entorno inherentemente no confiable: el navegador del usuario. Toda la lógica de autorización y validación de datos sensibles debe residir en el back-end. El front-end implementa medidas de seguridad como defensa en profundidad, nunca como única línea de protección.

### 14.2 Protección contra XSS (Cross-Site Scripting)

| Práctica                              | Regla                                                                                     |
|---------------------------------------|-------------------------------------------------------------------------------------------|
| <code>dangerouslySetInnerHTML`</code> | Prohibido salvo en casos excepcionales documentados con sanitización previa vía DOMPurify |
| Contenido dinámico del usuario        | Siempre renderizado como texto ( <code>{variable}`</code> ), nunca como HTML              |

|                |                                                                         |
|----------------|-------------------------------------------------------------------------|
| URLs dinámicas | Validar que comienzan con `https://` antes de usarlas en `href` o `src` |
| Cabecera CSP   | El servidor debe enviar una Content Security Policy estricta            |

```

...tsx
// Prohibido: vulnerable a XSS
<div dangerouslySetInnerHTML={{ __html: userProvidedContent }} />
// Correcto: con sanitización
import DOMPurify from 'dompurify';
const safeHtml = DOMPurify.sanitize(userProvidedContent);
<div dangerouslySetInnerHTML={{ __html: safeHtml }} />
// Mejor opción: renderizar como texto siempre que sea posible
<div>{userProvidedContent}</div>
...

```

### 14.3 Gestión Segura de Tokens de Autenticación

El almacenamiento de tokens de autenticación sigue una política estricta en Santo Pegasus:

| Método de Almacenamiento | Uso                             | Justificativa                                      |
|--------------------------|---------------------------------|----------------------------------------------------|
| `httpOnly` Cookie        | Refresh Token                   | Inaccesible para JavaScript; protege contra XSS    |
| Memoria (Zustand store)  | Access Token (corta duración)   | No persiste entre sesiones; limpiado al cerrar tab |
| `localStorage`           | Prohibido para tokens           | Accesible por cualquier script; vulnerable a XSS   |
| `sessionStorage`         | Prohibido para tokens sensibles | Accesible por JavaScript; inseguro contra XSS      |

### 14.4 Protección contra CSRF

- Las cookies de sesión deben ser emitidas por el back-end con los atributos `SameSite=Strict` o `SameSite=Lax` y `Secure`.
- Para operaciones mutantes (POST, PUT, DELETE), el back-end debe validar el origen de la petición.
- En formularios que requieran protección adicional, se debe usar el patrón de CSRF Token generado por el servidor.

### 14.5 Principio de Mínimo Privilegio en el Front-end

- No exponer información sensible en URLs: IDs de usuarios, tokens o datos de sesión no deben aparecer en parámetros de query string.

- **Feature flags por rol:** Los elementos de UI para acciones privilegiadas deben ocultarse para usuarios sin el rol correspondiente. Sin embargo, la autorización real siempre debe verificarse en el back-end.
- **Logging seguro:** Nunca registrar en ``console.error`` información sensible como tokens, datos personales o respuestas completas de la API.

---

## 15. CI/CD Front-end

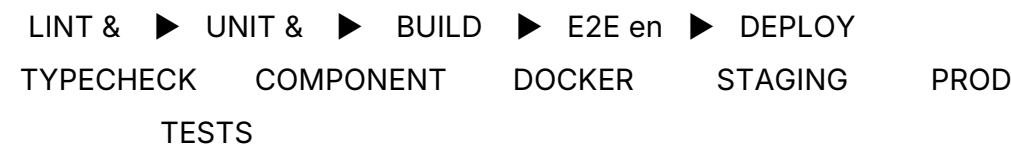
### 15.1 Filosofía del Pipeline

El pipeline de CI/CD del front-end debe garantizar que ningún código que falle en lint, pruebas o build llegue a producción. La automatización es la primera línea de control de calidad, complementando el proceso de Code Review humano.

### 15.2 Etapas del Pipeline

El pipeline de CI/CD se compone de las siguientes etapas ejecutadas secuencialmente:

...



...

### 15.3 Pipeline GitHub Actions — Definición Completa

```
```yaml
.github/workflows/ci.yml
```

```
name: CI/CD — Front-end

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [develop]

jobs:
  lint-and-typecheck:
    name: Lint & TypeCheck
```

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
- uses: actions/setup-node@v4

with:

node-version: '20'

cache: 'npm'

- run: npm ci
- run: npm run lint
- run: npm run typecheck

test:

name: Unit & Component Tests

needs: lint-and-typecheck

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
- uses: actions/setup-node@v4

with:

node-version: '20'

cache: 'npm'

- run: npm ci
- run: npm run test:coverage
- name: Upload Coverage Report
- uses: codecov/codecov-action@v4

build:

name: Docker Build

needs: test

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
- name: Build Docker Image
- run: docker build -t santopegasus/\${{ github.event.repository.name }}:\${{ github.sha }} .
- name: Push to Registry
- if: github.ref == 'refs/heads/develop' || github.ref == 'refs/heads/main'
- run: |
- docker push santopegasus/\${{ github.event.repository.name }}:\${{ github.sha }}

e2e:

name: E2E Tests (Playwright)

needs: build

runs-on: ubuntu-latest

if: github.ref == 'refs/heads/develop' || github.ref == 'refs/heads/main'

steps:

- uses: actions/checkout@v4
- uses: actions/setup-node@v4
with: { node-version: '20', cache: 'npm' }
- run: npm ci
- run: npx playwright install --with-deps
- run: npm run test:e2e

env:

PLAYWRIGHT_BASE_URL: \${ secrets.STAGING_URL }

- uses: actions/upload-artifact@v4

if: failure()

with:

name: playwright-report

path: playwright-report/

deploy-staging:

name: Deploy to Staging

needs: e2e

if: github.ref == 'refs/heads/develop'

runs-on: ubuntu-latest

steps:

- name: Deploy to Staging Environment
run: echo "Deploy automatizado al ambiente de Staging"

deploy-production:

name: Deploy to Production

needs: e2e

if: github.ref == 'refs/heads/main'

runs-on: ubuntu-latest

environment: production

steps:

- name: Deploy to Production
run: echo "Deploy a Producción con aprobación manual requerida"

...

15.4 Dockerfile para Front-end

El uso de Docker es obligatorio para garantizar la inmutabilidad del artefacto entre los ambientes de Staging y Producción, siguiendo el mismo principio establecido en la Guía de Ingeniería Back-end.

```
```dockerfile
```

### **Dockerfile — Multi-stage build para SPA con Vite**

```
FROM node:20-alpine AS builder
```

```
WORKDIR /app
```

```
COPY package.json ./
```

```
RUN npm ci --frozen-lockfile
```

```
COPY . .
```

```
RUN npm run build
```

### **Stage de producción con Nginx**

```
FROM nginx:1.25-alpine AS production
```

### **Copia la configuración de Nginx para SPA (manejo de rutas client-side)**

```
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

### **Copia el build generado por Vite**

```
COPY --from=builder /app/dist /usr/share/nginx/html
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

```
...
```

```
```nginx
```

nginx.conf — Configuración para SPA React

```
server {
```

```
    listen 80;
```

```
    root /usr/share/nginx/html;
```

```
    index index.html;
```

```
    # Redirecciona todas las rutas al index.html (client-side routing)
```

```
    location / {
```

```
        try_files $uri $uri/ /index.html;
```

```

}
# Cabeceras de seguridad
add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self'
'unsafe-inline';" always;

# Cache de assets estáticos (Vite genera hashes en los nombres)
location ~ \.(js|css|png|jpg|jpeg|webp|svg|ico|woff2)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
}
}
...

```

15.5 Variables de Entorno en el Front-end

Las variables de entorno en proyectos Vite deben seguir el prefijo `VITE\_` para ser expuestas al cliente. En proyectos Next.js, el prefijo es `NEXT\_PUBLIC\_`.

Reglas obligatorias:

- **Nunca incluir secretos en variables de entorno del front-end. Toda variable `VITE\_` o `NEXT\_PUBLIC\_` es embebida en el bundle y visible para cualquier usuario.**
- El archivo `.env.local` debe estar obligatoriamente listado en `.gitignore`.
- Las variables de entorno de Staging y Producción deben ser gestionadas en la plataforma de CI/CD (GitHub Actions Secrets, AWS Secrets Manager), nunca en archivos commiteados al repositorio.

```
```bash
```

## **.env.example — Archivo de referencia (commitado al repositorio)**

```
VITE_API_BASE_URL=https://api.santopegasus.com
```

```
VITE_APP_ENV=production
```

```
VITE_SENTRY_DSN=
```

## **.env.local — Configuración local del desarrollador (en .gitignore)**

```
VITE_API_BASE_URL=http://localhost:8080
```

```
VITE_APP_ENV=development
```

...

---

## Apéndice A — Checklist de Nuevo Proyecto

Utilice este checklist al iniciar un nuevo proyecto front-end en Santo Pegasus:

- ☐ Stack tecnológico configurado conforme a la Sección 3
- ☐ Estructura de carpetas conforme a la Sección 7.2
- ☐ ESLint y Prettier configurados con las reglas estándar (Sección 7.3 y 7.4)
- ☐ Husky + lint-staged + Commitlint configurados
- ☐ Cliente Axios centralizado con interceptores (Sección 6.2)
- ☐ TanStack Query Provider configurado en la raíz de la aplicación
- ☐ Stores Zustand organizados por dominio (Sección 5.2)
- ☐ Tailwind CSS con tokens de diseño personalizados (Sección 8.1)
- ☐ shadcn/ui instalado y componentes base en `src/components/ui/`
- ☐ Vitest configurado con umbral de cobertura del 80% (Sección 10.3)
- ☐ Playwright configurado para E2E
- ☐ Dockerfile multi-stage creado (Sección 15.4)
- ☐ Pipeline de CI/CD configurado (Sección 15.3)
- ☐ Variables de entorno documentadas en `.env.example`
- ☐ `README.md` con instrucciones de setup local

---

## Apéndice B — Glosario

Término	Definición
Atomic Design	Metodología de organización de componentes en átomos, moléculas, organismos, templates y páginas
Bundle	Archivo JavaScript resultante del proceso de build, enviado al navegador
CLS	Cumulative Layout Shift — métrica de estabilidad visual del layout
Code Splitting	Técnica de división del bundle en chunks cargados bajo demanda
CSP	Content Security Policy — política de seguridad HTTP que restringe recursos cargables
DTO	Data Transfer Object — objeto que define la forma de los datos en la comunicación con la API
E2E	End-to-End — pruebas que simulan el flujo completo del usuario en el navegador real

FCP	First Contentful Paint — primer momento en que el browser renderiza contenido
FID/INP	First Input Delay / Interaction to Next Paint — métricas de responsividad a la interacción
HMR	Hot Module Replacement — actualización de módulos en tiempo real durante el desarrollo
httpOnly Cookie	Cookie inaccesible para JavaScript, protegida contra XSS
JWT	JSON Web Token — estándar de token de autenticación stateless
LCP	Largest Contentful Paint — métrica de velocidad de carga del contenido principal
RSC	React Server Components — componentes renderizados en el servidor sin JS en el cliente
SSG	Static Site Generation — pre-renderización de páginas en build time
SSR	Server-Side Rendering — renderización de páginas en el servidor por request
XSS	Cross-Site Scripting — vulnerabilidad de inyección de scripts maliciosos

---

*Santo Pegasus Soluciones — Guía Oficial de Ingeniería Front-end v1.0.0 — Octubre 2025*

*Departamento de Ingeniería de Software / Chapter de Front-end*

*Este documento es revisado periódicamente. Consulte siempre la versión más reciente en el repositorio ``eng-guidelines``.*